

UniFlow

Production Readiness Roadmap

Technical Note · What to add to take UniFlow from prototype to production

UniFlow — Production Readiness Roadmap

What this is: UniFlow today is a working **v0 / MVP prototype** — multi-tenant, role-based, multi-industry, and AI-native, with the core flows working end-to-end. This document lists **what must be added to take it from prototype to production**, grouped by workstream and prioritized.

1. Where it stands today

Dimension	Status
Core features (retail + aviation, admin/employee)	[ok] Working
Multi-tenancy (per-company isolation)	[ok] Working
AI / MCP layer with per-brand key scoping	[ok] Working
Security hardening	[!] Prototype-level
Payments	[!] Simulated
Hosting / deployment	[x] Local only
Rate limiting / audit / governance	[x] Scaffolded, not wired
Automated tests / CI	[x] Minimal

The gap to production is **hardening, governance, payments, and deployment** — not core functionality.

2. Workstreams — what to add

2.1 Security & Authentication

- **Hash the MCP API keys** — store `sha256(key)` instead of plaintext, so a DB leak can't be used. (Small, high impact.)
- **Strong SECRET_KEY** from a secrets manager; rotate it; never commit it. Same for the Gmail and database credentials — move all secrets out of the repo `.env`.
- **Shorten the JWT lifetime** (currently ~14.6 days) to ~30–60 min, and add a **refresh-token** flow.
- **Enforce `is_active`** on login and on `get_current_user` — right now a disabled user can still log in (the "disable" feature has no real effect).
- **Whitelist the `role`** value on employee creation (currently unvalidated).
- **HTTPS/TLS** everywhere; secure headers; tighten CORS to real domains (currently hardcoded to `localhost:3000`).
- Consistent HTTP status codes (several "already exists / already done" cases return `404` instead of `409/400`).

2.2 AI / MCP Governance (*use the tables already scaffolded*)

The database already has `rate_limits`, `audit_logs`, and `mcp_tool_permissions` — they just need wiring. - **Require the `x-API-Key`** on the discovery endpoints (drop the no-key "all brands" fallback) — makes isolation enforced, not opt-in. (Or keep it deliberately as a public marketplace — a conscious choice, not a default.) - **Rate limiting** — per company / per tool / per window, returning `429`, so an AI client can't hammer the backend. - **Audit logging** — record every tool call and consequential action (who, what, when, result). - **Per-tool permissions** — let a company enable/disable individual MCP tools. - **Confirmation on write actions** (reserve / pay / cancel) — don't rely solely on the AI client's confirmation. - **(Larger, optional)** OAuth 2.1 + PKCE + dynamic client registration for a full standards-based auth, per the ChatGPT App Store spec.

2.3 Payments

- Replace the **simulated payment** with a real gateway — **Stripe** (or a local rail: **Chargily / SATIM / CIB**) — with webhooks, idempotency keys, and proper reconciliation of `payment_status`.

2.4 Reliability & Correctness (*fix the known bugs*)

- `email_status` tuple bug (trailing comma stores `("sent", )` instead of `"sent"`).
- `GET /bookings/` returns a **single** booking instead of the list.

- The sole-admin account-delete **cascade** misses `products/stores/inventory` (FK violation).
- `store` unique constraint typo (`__table_arg__`) — the (company, name) uniqueness is silently not created.
- `DELETE /inventory/{id}` returns a Python set instead of a JSON object.
- `resend-verification` inverted guard; reset email links to the verify page.
- **Booking seat race condition** — `available_seats` is read-then-decremented without a lock (can oversell). Use a row lock or atomic decrement.

2.5 Infrastructure & Deployment

- **Backend** -> a managed host (Cloud Run / Railway / Fly.io / AWS) + **managed PostgreSQL** (backups, PITR).
- **Frontend** -> Vercel (or equivalent).
- **MCP server(s)** -> hosted; decide the multi-tenant model (one MCP per brand, or one server that routes by the per-request key).
- Per-stage config (dev / staging / prod), environment-based secrets, HTTPS/CDN.

2.6 Observability

- Structured logging, error tracking (e.g. Sentry), uptime monitoring, and product analytics/metrics.

2.7 Testing & CI/CD

- Unit + integration tests (pytest) for the backend, component/E2E tests for the frontend.
- A CI pipeline: lint -> test -> run migrations -> deploy. (There's a `tests.py` but no real suite yet.)

2.8 Data & Migration hygiene

- Fix the recurring **Alembic alembic_version stamping** issue so upgrades are deterministic (no manual stamping). One migration head, tracked in CI.
- Separate demo **seed data** from production.

2.9 Documentation & Onboarding

- A company-facing onboarding guide: how to get your API key and connect your MCP app; API reference; SLAs.
-

3. Prioritized roadmap

- P0 — required before any real user touches it** 1. Hash API keys · strong secret · enforce `is_active` · secrets out of repo · HTTPS 2. Require the key + rate limiting + audit logging 3. Fix the critical correctness bugs (§2.4) 4. Deploy on real hosting + managed Postgres
- P1 — required to charge money / scale** 5. Real payments (§2.3) 6. Per-tool permissions + write-action confirmation 7. Observability + automated tests + CI/CD
- P2 — maturity / differentiation** 8. OAuth 2.1 + PKCE 9. MCP Apps interactive UI (rich in-chat experiences) 10. Horizontal scaling, multi-region, advanced governance

4. Effort snapshot

Item	Effort	Priority
Hash keys / secrets / <code>is_active</code> / HTTPS	S–M	P0
Require key + rate limit + audit (scaffolded tables)	M	P0
Critical bug fixes	S–M	P0
Hosting + managed DB	M	P0
Real payments + webhooks	M–L	P1
Tool permissions + write confirmation	M	P1
Observability + tests + CI	M–L	P1
OAuth 2.1 + PKCE	L	P2
MCP Apps UI	L	P2

S = days · M = 1–2 weeks · L = 3+ weeks (single developer, approximate).

5. The one-line summary

UniFlow's architecture is production-shaped; reaching production is about hardening (security,

governance), real payments, deployment, and testing — most of it building on structures (the multi-tenant model, the scaffolded governance tables) that already exist.